CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INTRODUCCIÓN A LA PROGRAMACION Y COMPUTACION 1
PRACTICA 1
SEGUNDO SEMESTRE 2026

Objetivos generales

- Familiarizar al estudiante con el lenguaje Java
- Aplicar conceptos aprendidos en clase magistral y laboratorio
- Desarrollar la lógica de programación y habilidades de diseño de algoritmos para la solución de los problemas planteados.

Objetivos específicos

- Desarrollar diagramas como parte del análisis de la aplicación solicitada.
- Construcción de algoritmos para cada proceso del sistema.
- Implementación de diferentes tipos de datos, arreglos y estructuras de control.
- Consolidar y profundizar el conocimiento de estructuras de control y procedimientos en JAVA.
- Construcción de una aplicación simple en consola.

Descripción de la actividad

La empresa de desarrollo Coden' Bugs lo contrata para crear un programa que simula un arcade de juegos sencillos en consola. El arcade ofrecerá 3 juegos para que el usuario pueda jugar, debe ser intuitivo y amigable con el usuario permitiendo la navegación entre juegos y reportes con menús e instrucciones claras. Además deberá recopilar datos sobre las partidas jugadas para poder generar con ellos estadísticas y reportes globales que el usuario pueda consultar.

Juego 1: Adivina anagramas

El juego consiste en formar tantas palabras como sea posible usando las letras de una palabra dada.

Condiciones iniciales

El juego tendrá un **banco de palabras** válidas previamente definido por el desarrollador, todas las palabras deben tener al menos tres letras, sin caracteres especiales ni espacios y sin diferenciar entre mayúsculas y minúsculas, notar que estas condiciones también aplican a las palabras ingresadas por el jugador al intentar adivinar.

En base al banco de palabras definido, el programa deberá calcular automáticamente el número de anagramas correctos por conjunto de letras, es decir, la cantidad de palabras que tienen exactamente las mismas letras pero en diferente orden.

El desarrollador también definirá un porcentaje global de aceptación de fallos, este porcentaje servirá para calcular el número de intentos fallidos posibles a partir del número de anagramas correctos.

$$Intentos\ fallidos\ permitidos\ = floor(anagramas_correctos\ *\ porcentaje_aceptación)$$

Si este número fuera cero, quiere decir que al primer intento fallido perderá el juego.

Desarrollo del juego

Del banco se seleccionará una palabra aleatoria y se le presentará al usuario con las letras mezcladas de forma desordenada.

El jugador tiene que formar palabras válidas, que sí existan en el banco de palabras y usando todas las letras de la palabra desordenada que se le presentó, sin quitar ni agregar ninguna letra.

Si el jugador ingresa una palabra que ya había ingresado previamente se le indica que debe ser diferente y no se registra como intento, por lo que tampoco se registra como acierto o error.

Si el jugador adivina todos los anagramas posibles para el conjunto de letras dado habrá ganado la partida.

Es importante aclarar que la palabra original que se seleccionó aleatoriamente del banco de palabras (la que se desordenó para presentar al usuario) debe ser adivinada también, se toma solo para obtener el conjunto de letras desordenadas, no se excluye de las palabras que deben ser adivinadas.

Si el jugador ingresa una palabra que no es una permutación de las letras dadas o no está registrada en el banco de preguntas se considera como un intento fallido y se gasta una de las oportunidades de intento fallido, si se supera la cantidad de intentos fallidos posibles habrá perdido la partida.
En cada intento se debe mostrar el progreso y estado de la partida, incluyendo el conjunto de letras dado al inicio, el listado de palabras adivinadas, el listado de palabras fallidas y la cantidad de intentos fallidos restantes.

Ejemplo

Supongamos que se tiene un banco de preguntas pequeño:

{GATO, TOGA, GOTA, SON, NOS}

En este caso 'GATO', 'TOGA' Y 'GOTA' se forman con las mismas letras, quiere decir que el número posible de anagramas correctos para cualquiera de ellas es 3, del mismo modo para 'SON' y 'NOS' sería 2.
Además digamos que el porcentaje de aceptación será 70%, de modo que para cualquiera de las primeras 3 palabras se tendrán 2 oportunidades en las que el jugador puede ingresar una palabra incorrecta, al ingresar una 3era palabra incorrecta perderá el juego. Por el contrario si logra adivinar las 3 palabras correctas antes, ganará el juego

Juego 2: Tic Tac Toe

El juego consiste en el clásico tres en raya, jugado por el usuario contra la computadora.

Condiciones iniciales

El tablero es una cuadrícula de 3x3.
Para que el jugador pueda indicar en qué celda desea colocar su ficha, cada celda debe tener un nombre único, combinando un **identificador** de columna y uno de fila, para ello se recomiendan números o letras pero queda a criterio del desarrollador, por ejemplo si usamos letras para las columnas y números para las filas, la celda superior izquierda se identificaría como A1, la celda central sería B2, y así sucesivamente.

El jugador siempre inicia la partida y la computadora juega el segundo turno de cada ronda, el desarrollador puede definir qué ficha o caracter usa cada jugador

Desarrollo del juego

Al inicio de cada turno se debe mostrar el tablero completo. En las celdas vacías se debe mostrar el identificador de la celda para que el jugador sepa qué escribir, y en las celdas ocupadas se debe mostrar la ficha correspondiente.

El jugador indica en qué celda desea jugar escribiendo su coordenada (por ejemplo, "B2"). El programa debe validar que la coordenada sea válida y que la celda está vacía; si la jugada no es válida, se le debe indicar el error al jugador y solicitarle que ingrese una nueva coordenada.

Después de cada jugada, se debe revisar si existe una condición de victoria o empate. Si la partida no ha terminado, le toca el turno al siguiente jugador.

La computadora debe elegir automáticamente una celda vacía disponible para jugar. Como mínimo, se acepta que la computadora elija su jugada de forma aleatoria entre las celdas disponibles.

Habrá una **bonificación** de hasta el 5% de la nota si se logra implementar y defender un algoritmo que evalúe el estado del tablero y decida la mejor jugada posible, por ejemplo, ganar si existe una jugada ganadora disponible, bloquear al jugador si este está a punto de ganar, o priorizar posiciones ventajosas como el centro o las esquinas. Considerar que esta bonificación será extra por lo que no es obligatorio y que la nota final de la práctica no puede exceder el 100%.

Al finalizar la partida se debe mostrar el tablero final junto con el resultado (victoria del jugador, victoria de la computadora, o empate).

Juego 3: Sudoku

El juego consiste en el clásico sudoku, jugado en un tablero de 9x9 dividido en 9 subcuadros (cajas) de 3x3.

Condiciones iniciales

El tablero inicial debe generarse de forma aleatoria en cada partida, y debe cumplir las siguientes condiciones:

- Debe respetar las reglas del sudoku: ningún número del 1 al 9 puede repetirse dentro de una misma fila, columna, o caja de 3x3.
- Debe tener al menos una solución posible, no se exige que la solución sea única, basta con que el tablero generado sea válido y se pueda resolver.
- Debe tener una cantidad fija de celdas iniciales llenas, que son las “pistas" iniciales, definida por el desarrollador, se recomienda alrededor de 35 celdas llenas de las 81 totales, pero nuevamente queda a criterio del desarrollador. Importante considerar que estas celdas que forman parte del tablero inicial son fijas y no pueden ser modificadas por el jugador durante la partida.

Al igual que en Tic Tac Toe, cada celda debe poder identificarse con un nombre o coordenadas para que el jugador indique sus movimientos; nuevamente se recomienda usar el formato “fila,columna”, por ejemplo, "3,5" para la celda en la fila 3, columna 5.

Desarrollo del juego

Al inicio de la partida se debe mostrar el tablero completo, mostrando los valores de las celdas fijas y las vacías, organizado visualmente de forma que se distingan las 9 cajas de 3x3.
El jugador indica la celda donde desea colocar un número y el valor que desea colocar. El programa debe validar que la jugada sea válida, esto implica:

- Que la coordenada sea válida
- Que la celda indicada no sea una celda fija del tablero inicial
- Que el valor ingresado sea válido
- Que no se rompa ninguna regla del sudoku, es decir que no se repita en la misma fila, columna o caja de 3x3

Si alguna de estas validaciones falla, se le debe indicar al jugador el motivo por el cual su jugada no es válida, y se le debe permitir intentar de nuevo.
Se debe permitir que el jugador pueda cambiar el valor de una celda que él mismo llenó anteriormente para que pueda corregir sus propias jugadas.
Después de cada jugada válida se debe mostrar el tablero actualizado. En todo momento se debe poder abandonar el juego y regresar al menú, con ello se perderá el progreso de la partida actual, al volver a ingresar se generará un nuevo tablero.

El jugador gana la partida cuando todas las celdas del tablero están llenas y no existe ninguna violación de las reglas del sudoku en ninguna fila, columna o caja.

Reportes

El jugador debe poder consultar varias estadísticas globales en el arcade, estas son:

Anagramas:

- Cantidad de partidas jugadas
- Cantidad total de palabras adivinadas y total de intentos fallidos
- Porcentaje general de victorias

Tic Tac Toe:

- Cantidad de victorias del jugador, de la máquina y empates
- Racha actual de victorias y derrotas consecutivas
- Promedio de jugadas por partida

Sudoku:

- Cantidad de partidas ganadas y abandonadas
- Promedio de jugadas inválidas por partida
- Record de menor número de jugadas para ganar una partida

Importante

- Usar lenguaje de programación JAVA
- Práctica obligatoria para tener derecho al siguiente proyecto.
- Aplicación exclusivamente en consola.
- Es válido utilizar algún IDE o cualquier editor de texto.
- **Copias obtendrán nota de CERO, se notificará a coordinación.**
- **Copiar código de internet o generado con IA sin entender el funcionamiento se penalizará con nota de cero**
- **Entender la funcionalidad de la totalidad del código para que se tome como válido.**

Entrega

La fecha de entrega es el 17 de agosto en la publicación correspondiente de Classroom. Los componentes a entregar son:

- Código fuente.
- Código compilado (ejecutable).
- Manual técnico, incluyendo algoritmos del flujo general del programa y de los subprogramas más importantes.
- Manual de usuario

Para la calificación el estudiante debe llevar impresa la hoja de calificación que se publicará en los siguientes días.